Project Report

Distilling Vision Models to LiDAR Data for Cross-Modal Perception in Autonomous Systems

Team Decepticons

Krishna Goel (2023112009)
Dhruv Bansal (2023102048)

December 6, 2025

Contents

1 Project Description **3**

2 Project Objectives **3**

3 RGB–LiDAR Alignment: Theory and Sequential Implementation **4**
3.1 Loading Sensor Data 4
3.2 Constructing Homogeneous Transformation Matrices 4
3.3 Final LiDAR→Camera Transformation 4
3.4 Transforming LiDAR Points to the Camera Frame 5
3.5 Projecting Points Onto the Image Plane 5
3.6 Importance of the Alignment 5
3.7 Plots and Visualisations 6

4 Teacher Feature Extraction, Student Training, and t-SNE Visualization **7**
4.1 Loading the Teacher Model (DINOv2) 8
4.2 Preparing Images for the Teacher Model 8
4.3 Building the Student Model (LiDAR Network) 8
4.4 Training the Student Using Distillation Loss 9
4.5 Quantitative Evaluation with Cosine Similarity 9
4.6 Visualizing Feature Alignment with t-SNE 9

5 Exploratory Attempt: Semantic Segmentation Head **10**
5.1 Challenges and Limitations 10

6 Real-Life Applications **10**
6.1 Autonomous Systems 10
6.2 Robotics and Industrial Use 11

7 Future Scope **11**
7.1 Advanced Distillation Methods 11
7.2 Scaling and Downstream Tasks 11
7.3 Real-World Robustness 11

8 Final Conclusion **12**

1 Project Description

This project aims to teach a LiDAR-only model to understand high-level scene semantics by learning from a vision model (like CLIP or DINOv2). Using paired RGB and LiDAR data, the vision model acts as a "teacher," providing rich features from images. The LiDAR model, the "student," is trained to match these features. After training, the LiDAR model can recognize semantic information without needing images.

2 Project Objectives

1. **Literature Review:**
   Study existing multi-modal distillation and 2D–3D representation transfer methods (e.g., UniDistill, LabelDistill, DistillBEV).

2. **Dataset Preparation:**
   Load and preprocess a small subset of KITTI or nuScenes containing paired RGB images and LiDAR point clouds.

3. **Teacher Model Setup:**
   Use a pretrained vision foundation model (e.g., CLIP or DINOv2) to extract semantic features from images.

4. **Student Model Design:**
   Build a lightweight LiDAR-only network (e.g., simplified PointNet) to map point cloud inputs to feature vectors.

5. **Cross-Modal Distillation:**
   Train the student model to minimize MSE loss between LiDAR features and teacher vision embeddings.

6. **Evaluation and Visualization:**
   Analyze feature similarity (e.g., via cosine distance and t-SNE plots) before and after distillation to demonstrate alignment improvements.

7. **Reporting and Presentation:**
   Document the findings, results, and possible extensions for future work.

3 RGB–LiDAR Alignment: Theory and Sequential Implementation

To train a LiDAR-only model using cross-modal distillation, the LiDAR point cloud and RGB image must describe the scene from the same viewpoint. Since the sensors are mounted at different positions and orientations, their raw outputs do not align. The goal of this stage is to convert all LiDAR points into the camera frame and project them onto the 2D image plane, completing the dataset preparation step.

3.1 Loading Sensor Data

For each sample, we load:

- The RGB image:
$$\text{Image} \in \mathbb{R}^{H \times W \times 3},$$
- The LiDAR point cloud:
$$\text{pts} \in \mathbb{R}^{N \times 3},$$
where each row (x, y, z) is a point in the LiDAR frame.

At this point, both sensors exist in different coordinate systems.

3.2 Constructing Homogeneous Transformation Matrices

The nuScenes dataset provides calibration for each sensor, including:

- A rotation quaternion $\rightarrow$ rotation matrix $R \in \mathbb{R}^{3 \times 3}$,
- A translation vector $t \in \mathbb{R}^{3 \times 1}$.

These form a homogeneous transform:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}.$$

For a homogeneous point $p = [x, y, z, 1]^T$, the transformed point is:

$$p' = T\,p.$$

Using calibration and ego-pose data, we obtain four transforms, each 4×4:

$$T_{\text{lidar}\rightarrow\text{ego}}, \quad T_{\text{ego(lidar)}\rightarrow\text{world}}, \quad T_{\text{world}\rightarrow\text{ego(camera)}}, \quad T_{\text{ego}\rightarrow\text{cam}}.$$

3.3 Final LiDAR→Camera Transformation

To express any LiDAR point in the camera's coordinate frame, we combine the transforms in order:

$$T_{\text{lidar}\rightarrow\text{cam}} = T^{-1}_{\text{cam}\rightarrow\text{ego}}\, T^{-1}_{\text{world}\rightarrow\text{ego(camera)}}\, T_{\text{ego(lidar)}\rightarrow\text{world}}\, T_{\text{lidar}\rightarrow\text{ego}} \in \mathbb{R}^{4 \times 4}.$$

This matrix accounts for all rotations, translations, and timing differences between sensors.

3.4 Transforming LiDAR Points to the Camera Frame

We convert the LiDAR points into homogeneous form:

$$\text{pts}_h = [\text{pts} \quad \mathbf{1}] \in \mathbb{R}^{N\times 4}.$$

Applying the full transformation:

$$\text{pts}_{\text{cam}} = (T_{\text{lidar}\to\text{cam}} \cdot \text{pts}_h^T)^T \in \mathbb{R}^{N\times 4}.$$

The 3D camera-frame coordinates are:

$$(x_{\text{cam}}, y_{\text{cam}}, z_{\text{cam}}) \in \mathbb{R}^{N\times 3}.$$

Points with $z_{\text{cam}} \leq 0$ (behind the camera) are removed.

3.5 Projecting Points Onto the Image Plane

The camera intrinsic matrix is:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 3}.$$

We stack the camera-frame points:

$$\text{pts}_{\text{cam-3D}} = \begin{bmatrix} x_{\text{cam}} \\ y_{\text{cam}} \\ z_{\text{cam}} \end{bmatrix} \in \mathbb{R}^{3\times N}.$$

Projection gives:

$$\begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = K \begin{bmatrix} x_{\text{cam}} \\ y_{\text{cam}} \\ z_{\text{cam}} \end{bmatrix}.$$

Pixel coordinates are:

$$u = \frac{u'}{w'}, \quad v = \frac{v'}{w'}.$$

The final valid projections satisfy:

$$0 \leq u < W, \quad 0 \leq v < H,$$

producing

$$(u, v) \in \mathbb{R}^{N_{\text{valid}}\times 2}.$$

3.6 Importance of the Alignment

These pixel coordinates show exactly where each LiDAR point appears in the image. When plotted, they align with real objects like cars, roads, and buildings, confirming that the geometric transformations are correct.

3.7 Plots and Visualisations

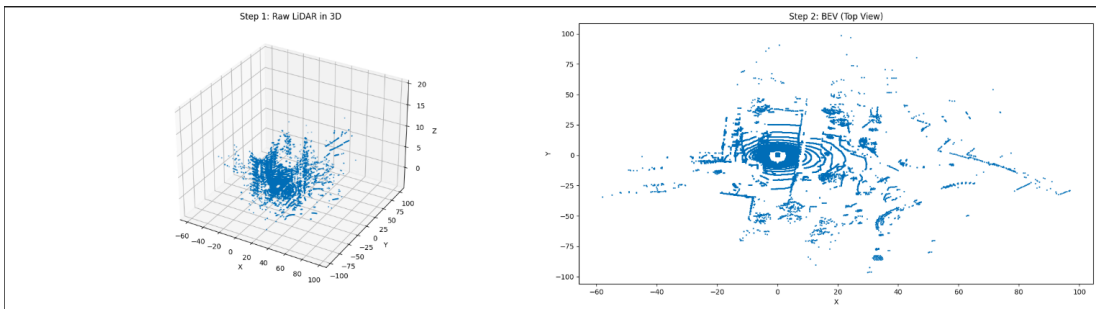

Figure 1: Image 1 (Step 1) shows the raw LiDAR points in 3D as captured by the sensor, while Image 2 (Step 2) shows the same points from a top-down bird's-eye view to clearly reveal the scene layout and verify that the LiDAR data is correctly loaded before any transformations.

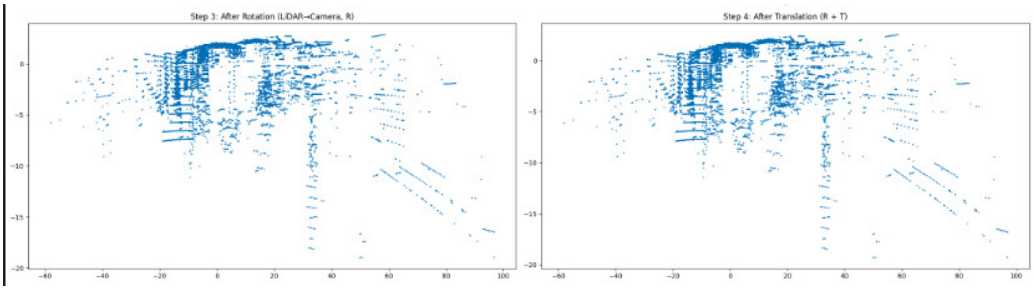

Figure 2: Image 3 (Step 3) shows the LiDAR points after rotation to match the camera's viewing direction, while Image 4 (Step 4) shows the same points after applying both rotation and translation to place them at the camera's actual position, completing the 3D alignment into the camera frame.

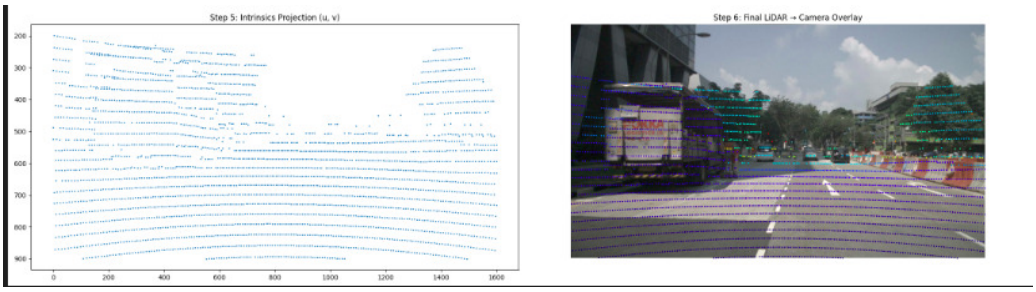

Figure 3: Image 5 shows the LiDAR points after being mathematically projected into 2D pixel coordinates using the camera intrinsics, while Image 6 displays these projected points overlaid on the actual camera image, confirming that the LiDAR–camera alignment is correct and that the points fall exactly on visible scene objects.

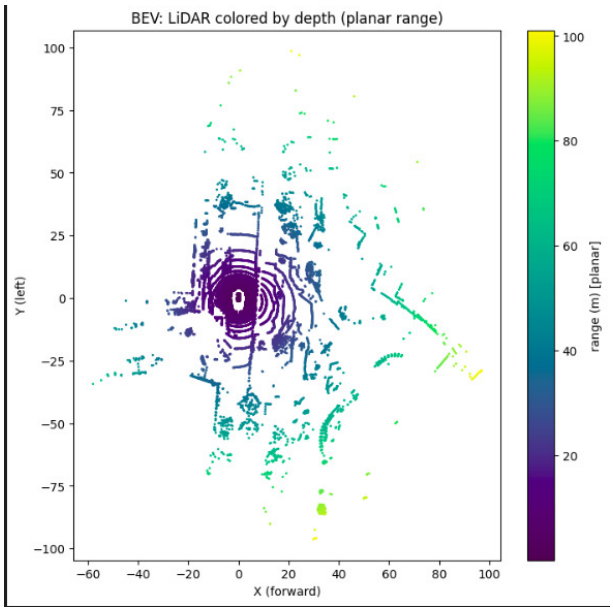

Figure 4: This picture shows the LiDAR points from a top-down view, like looking at the scene from above. Each point is colored by how far it is from the LiDAR sensor—nearby points are darker, and faraway points are lighter.

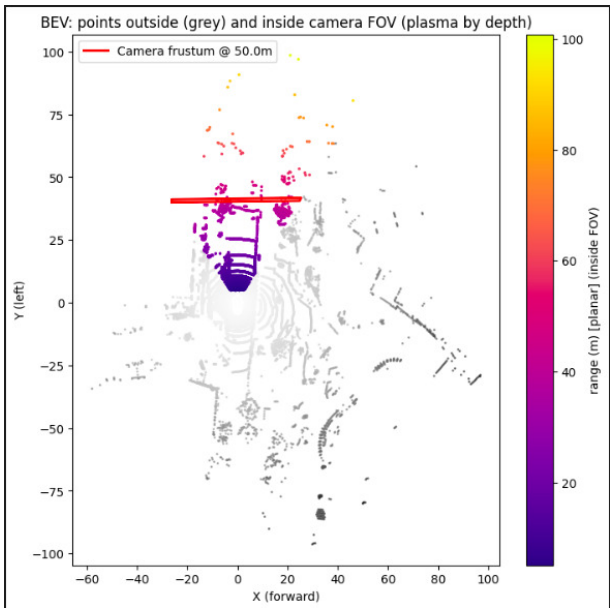

Figure 5: This image shows the same top-down LiDAR view, but now the points are split into grey (outside the camera's field of view) and colored/plasma (inside the camera's field of view). The red polygon marks the camera's approximate viewing cone at 50 meters, showing which LiDAR points the camera can actually see.

4 Teacher Feature Extraction, Student Training, and t-SNE Visualization

Once the LiDAR points are aligned with the camera coordinate system, the next phase of the pipeline is to teach a LiDAR-only network to learn semantic representations from

a vision model. For this purpose, we utilize DINOv2 as the "teacher" and a lightweight Multilayer Perceptron (MLP) as the "student."

The objective is to ensure that whenever both sensors observe the same scene geometry, the student produces feature vectors that closely resemble those produced by the teacher. The following subsections detail the implementation sequence.

4.1 Loading the Teacher Model (DINOv2)

We utilize the pretrained DINOv2 model, a vision transformer trained on a massive dataset to produce robust semantic features.

- The model is loaded in evaluation mode to freeze its weights.
- We verify that for a given image input, the model outputs a feature vector of dimension $D = 384$ per patch.
- We confirm that gradients are enabled for the student but disabled for the teacher, as the teacher's weights remain static.

4.2 Preparing Images for the Teacher Model

To extract features, the RGB images must be preprocessed to match the teacher's expected input format:

1. The image tensor is resized such that both dimensions are multiples of the patch size (14 pixels).
2. Standard ImageNet normalization is applied.
3. The image is passed through DINOv2, which divides the image into 14×14 patches.

This results in a set of patch tokens, where each token is a 384-dimensional vector. Since we previously calculated the pixel coordinates (u, v) for every LiDAR point, we can now map each 3D point to a specific image patch. The feature vector of that patch is assigned as the **target feature** (f_{teacher}) for the corresponding LiDAR point.

4.3 Building the Student Model (LiDAR Network)

The student model is designed as a simple MLP that processes raw 3D coordinates.

- **Input:** A LiDAR point $p = (x, y, z) \in \mathbb{R}^3$.
- **Architecture:** A sequence of linear layers with ReLU activations:

$$3 \to 128 \xrightarrow{\text{ReLU}} 256 \xrightarrow{\text{ReLU}} 384$$

- **Output:** A feature vector $f_{\text{student}} \in \mathbb{R}^{384}$.

The output dimension is strictly matched to the teacher's output to facilitate direct comparison.

4.4 Training the Student Using Distillation Loss

The training process minimizes the difference between the student's predicted features and the teacher's extracted features. For a batch of N points, we compute the Mean Squared Error (MSE) loss:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \| f_{\text{student}}^{(i)} - f_{\text{teacher}}^{(i)} \|_2^2$$

During training, the student updates its weights via backpropagation to minimize $\mathcal{L}$. The training logs indicate a steady decrease in loss, confirming that the student is successfully approximating the teacher's representation space.

4.5 Quantitative Evaluation with Cosine Similarity

Post-training, we evaluate the alignment using Cosine Similarity, which measures the cosine of the angle between two vectors:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

We compared the teacher's features against the student's features before and after training:

- **Untrained Student vs. Teacher:** Similarity ≈ 0.009 (implies no correlation).
- **Trained Student vs. Teacher:** Similarity ≈ 0.69 (implies strong alignment).

This significant increase confirms that the distillation process was effective.

4.6 Visualizing Feature Alignment with t-SNE

To interpret the high-dimensional (384-D) feature space, we utilize t-SNE (t-Distributed Stochastic Neighbor Embedding) to project the vectors into 2D. The resulting plot visualizes three distributions:

- **Yellow:** Teacher features (Ground Truth).
- **Teal:** Untrained student features.
- **Purple:** Trained student features.

Observation: The plot reveals that the untrained student's features form a distinct, scattered cluster far from the teacher. However, the trained student's features (purple) shift significantly to overlap with the teacher's cluster (yellow). This visual evidence demonstrates that the LiDAR network has successfully learned to encode geometric inputs into a semantic feature space similar to that of the vision model.

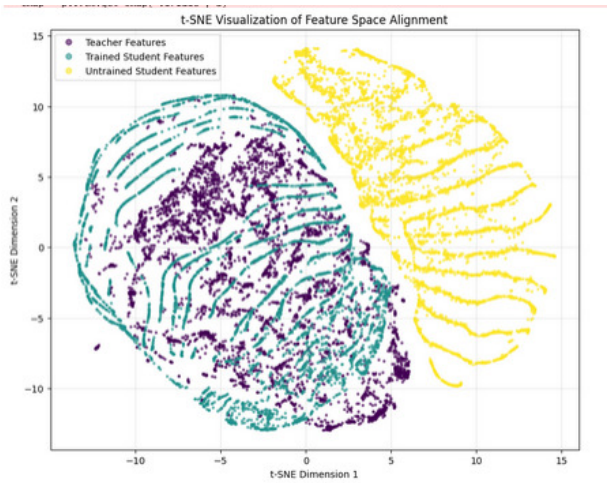

Figure 6: The t-SNE plot shows that the trained student's features (purple) move much closer to the teacher's features (yellow), while the untrained student's features (teal) stay far apart, proving successful feature alignment after distillation.

5 Exploratory Attempt: Semantic Segmentation Head

We also tried going a step beyond the main distillation task by adding a small semantic segmentation head on top of the frozen student model. The idea was to use the 384-dimensional features learned from the DINOv2 teacher to predict semantic labels for each LiDAR point in the nuScenes dataset. We reused our LiDAR-to-camera projection pipeline and attached a lightweight classifier that attempted to map the student's features to class labels.

5.1 Challenges and Limitations

While the concept was straightforward, the implementation ran into alignment problems. After projection and downsampling, the LiDAR points no longer matched their corresponding semantic labels, meaning the model received features from one set of points but labels from another. This mismatch led to shape errors and made training impossible. Fixing this would require a more advanced point-to-label synchronization pipeline, which was beyond the original scope. Due to time constraints, we were unable to continue refining this part, so the segmentation module remains an incomplete exploratory experiment.

6 Real-Life Applications

6.1 Autonomous Systems

All-Weather Autonomous Driving (L3+). Our distilled LiDAR model can support driving in low-light or poor weather conditions. When cameras fail due to darkness, fog, or glare, LiDAR semantics remain stable. This improves safety and makes the system more robust.

Edge Computing Deployment. The student model is light and fast, so it can run on low-power embedded systems used in real vehicles. This makes deployment practical in real-world setups.

Real-Time Semantic Mapping. The model can classify LiDAR points in real time, which helps build accurate maps with semantic meaning. These maps help in localization, navigation, and high-level planning.

6.2 Robotics and Industrial Use

Industrial Inspection. Mobile robots can use the distilled features to identify equipment, track inventory, or spot defects in cluttered industrial spaces.

Construction Monitoring. Fast point cloud understanding helps identify building materials, track progress, and flag possible safety issues on construction sites.

Perimeter Security. The model can classify objects reliably (for example, person vs. vehicle) even in difficult conditions. This reduces false alarms that occur with simple motion-based systems.

7 Future Scope

7.1 Advanced Distillation Methods

Relational Distillation. Instead of matching individual features only, the student can also learn the relationships between features. This can help produce a more consistent LiDAR feature space.

Attention-Guided Focus. Teacher model attention maps can guide the LiDAR student to focus on the most important points in the scene.

BEV Alignment. Projecting features to a Bird's-Eye-View grid can improve alignment between 2D and 3D spaces and enhance geometric consistency.

7.2 Scaling and Downstream Tasks

Downstream Validation. Future work can use the distilled features for tasks like 3D object detection and 3D semantic segmentation to show their full value.

Open-Vocabulary Perception. By using vision-language model (VLM) features, the LiDAR model could recognise new object categories without retraining.

Full Dataset Training. Training on the complete KITTI or nuScenes dataset can test how well the model handles generalization in large-scale conditions.

7.3 Real-World Robustness

Adversarial Evaluation. The system can be tested under challenging camera conditions such as fog, night, or heavy shadows to check if the LiDAR student still performs well.

Model Compression. Further compressing the student network can reduce computation and memory use, making deployment easier on edge devices.

8 Final Conclusion

This project set out to understand whether a LiDAR model can learn rich semantic information by distilling knowledge from a powerful image-based teacher model. We built the full pipeline—from projecting LiDAR points into the camera frame, to extracting patch-level features from DINOv2, to training a lightweight student network to mimic these features. The results clearly showed successful knowledge transfer: the trained student's embeddings aligned closely with the teacher's, both quantitatively through cosine similarity and visually through t-SNE. This confirms that LiDAR can learn meaningful, image-driven feature spaces without needing labels. Although we attempted to extend the work toward semantic segmentation, time constraints and alignment issues prevented this part from being fully completed. Overall, the project demonstrates a solid proof-of-concept for cross-modal distillation and opens the door for deeper experiments in LiDAR-only semantic understanding.